The pxUML package

UML class diagrams (pgf-umlcd) for P3CTeX / UOC

Scvria
P3CTeX / UOC PEC Repository

2025/02 – Version 0.1

Abstract

pxUML provides a LaTeX2e API for UML class and object diagrams on top of pxpgf-umlcd (and TikZ). Class definitions are stored in an internal registry (via pxPRP); drawing is separate from definition, so the same logic can be reused and layout adjusted without changing the model. It is tailored for the UOC use case (e.g. PEC diagrams).

Contents

1 Introduction **1**

2 Loading the package **2**

3 User command reference **2**

3.1 Class definition 2

3.2 Extending a class 2

3.3 Drawing 3

3.4 Inspection 3

4 Minimal example **3**

5 Change history **5**

1 Introduction

pxUML keeps a *class registry*: each class has a name, attributes, operations, inheritance, object type (e.g. class, interface, enumclass), visibility, and optional implementation. You define classes with \PXdefClass and optionally extend them with \addAttribute, \addOperation, and \addInheritance. Drawing commands (\drawclass, \drawsimpleclass, \drawInstance) read from this registry, so you can change layout without redefining the class.

2 Loading the package

```
\usepackage{pxUML}
```

The package requires `expl3`, `xparse`, `l3keys2e`, `color`, `pxpgf-umlcd`, `pxPRP`, and `pgf-umlsd`.

3 User command reference

3.1 Class definition

`\pxUMLClass` Defines or overwrites a class in the registry. The starred form sets visibility to "private" (e.g. for diagram styling).

```
\pxUMLClass[*][objtype]{ClassName}[inherit]{attributes}[operations]<implementation>
```

- `<objtype>`: `class` (default), `interface`, `enumclass`, etc., as in the underlying pgf-umlcd environment.
- `<attributes>`: comma-separated list of attribute strings (e.g. `- name : String`).
- `<operations>`: optional; comma-separated list of operation strings.
- `<implementation>`: optional; implementation class for interfaces.

Example: `\PXdefClass{MyClass}{- id : int}{[+ get() : int]}`.

3.2 Extending a class

`\addAttribute` Appends an attribute to the list of attributes of an existing class.

```
\addAttribute{<ClassName>}{<attribute>}
```

`\addOperation` Appends an operation to the list of operations of an existing class.

```
\addOperation{<ClassName>}{<operation>}
```

`\addInheritance` Adds an inheritance (parent) to the class. The second argument is the *parent class name* only (as expected by the underlying `\inherit` of pgf-umlcd).

```
\addInheritance{<ClassName>}{<ParentClassName>}
```

Example: `\addInheritance{DerivedClass}{BaseClass}`.

3.3 Drawing

`\drawclass` Draws a full class box (attributes and operations) from the registry at the given TikZ coordinates. Optional first argument `[...]` is passed as TikZ options (e.g. `text width=14em`). Starred form uses the package "variable" colour scheme for that class.

```
\drawclass[*][<options>]{<ClassName>}[<x,y>]
```

`\drawsimpleclass` Same as `\drawclass` but draws only the simple class form (attributes and inheritance, no operations list). Same optional behaviour for star and options.

```
\drawsimpleclass[*][<options>]{<ClassName>}[<x,y>]
```

`\drawinstance` Draws an object instance (lifeline-style box) of a class, with explicit attribute and optional operation list.

```
\drawinstance[*][<text width em>]{<InsName>}[<x,y>]{<ClassName>}{<attr list>}[<op list>]
```

Default text width is 16 em. Starred form uses the variable colour scheme.

`\pxUMLDiagram` Puts the given TikZ/pgf-umlcd content inside a resized box and applies the package colour scheme. Starred form uses the variable colour scheme for the whole diagram. Optional width defaults to `.8\linewidth`. Optional size for horizontal space before diagram defaults to `-2em`.

```
\pxUMLDiagram[*][<tikz options>]{<content>}[<width>]<hspace>
```

Example: `\PXuml{\drawclass{MyClass}}[1\linewidth]`.

3.4 Inspection

`\PXshowclass` Loads the stored data for the class into internal token lists (for debugging or further macros). If the class does not exist in the registry, all keys are effectively empty; no error is raised.

```
\PXshowclass{<ClassName>}
```

4 Minimal example

```
\usepackage{pxUML}

\begin{document}

%DEFINE CLASSES
 \pxUMLClass{Person}[Animal]{- name : String}[+ getName() : String]
 \pxUMLClass[abstractclass]{Animal}{- name : String}[+ getName() : String]
 \pxUMLClass[enumclass]{EyeColor}{Brown, Blue, Green}
 \pxUMLClass[interface]{Science}{- id : int}[+ solveProblem(): int]
```

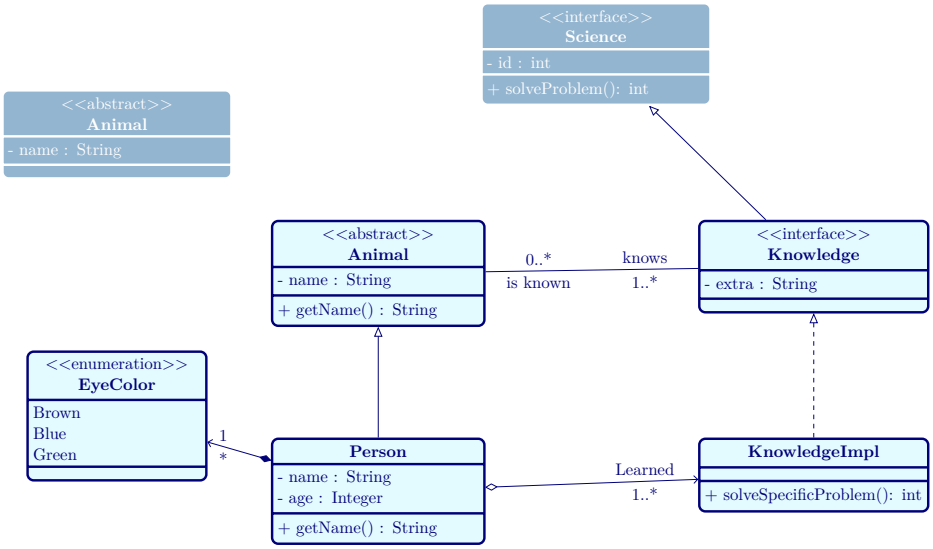
```
 \pxUMLClass[interface]{Knowledge}{- extra : String}
 \pxUMLClass{KnowledgeImpl}{}[+ solveSpecificProblem(): int]<Knowledge>
%ADD NEW PROPERTIES
 \addAttribute{Person}{- age : Integer}
 \addInheritance{Knowledge}{Science}
%DRAW DIAGRAM
 \pxUMLDiagram{

  \drawsimpleclass*{Animal}[-6,3]
  \drawclass[text width=12em]{Animal}[0,0]
  \drawclass[text width=12em]{Person}[0,-5]
  \drawclass[text width=10em]{EyeColor}[-6,-3]

  \drawclass*{Science}[5,5]
  \drawsimpleclass{Knowledge}[10,0]
  \drawclass{KnowledgeImpl}[10,-5]
  %%  RELATIONS
  \association{Animal}{0..*}{is known }{Knowledge}{knows}{1..*}
  \composition{Person}{1}{*}{EyeColor}
  \aggregation{Person}{Learned}{1..*}{KnowledgeImpl}

 }[.9\linewidth]<-4em>

\end{document}
```



5 Change history

v0.1 Initial release: class registry, `\PXdefClass`, `\drawclass`, `\drawsimpleclass`, `\drawInstance`, `\PXuml`, `\PXshowclass`, `\addAttribute`, `\addOperation`, `\addInheritance`. Documentation and tests aligned with pxPRP workflow.